

Lawyer Finder

Complete Codebase Guide for Beginners

CSE 3104 - Database Systems Lab Project

Ahsanullah University of Science and Technology

About This Document

This guide is written for **absolute beginners**. It walks you through every single file in the Lawyer Finder codebase -- from the database schema to the PHP backend API to the React frontend -- explaining *what* each piece does, *why* it exists, and *how* it connects to the rest of the system.

If you have never built a full-stack web application before, this is the document you need. Every SQL query, every PHP endpoint, and every React component is explained in plain English with real code snippets from the project.

Metric	Value
Total Backend PHP Files	10 files across 3 directories
Total Frontend React Files	20+ components and pages
Database Tables	3 (users, lawyers, appointments)
SQL Operations Demonstrated	Transactions, Locks, JOINS, Aggregates, Soft-Delete
Authentication	PHP Sessions + bcrypt password hashing
Design System	Google Material Express 3 tokens
How to Run	python run.py (one command)

Table of Contents

- 1. What is Lawyer Finder? (Project Overview)**
- 2. High-Level Architecture (How the Three Layers Work Together)**
- 3. Project Directory Structure (File Map)**
- 4. Database Schema Deep Dive (schema.sql)**
 - 4.1 The users Table
 - 4.2 The lawyers Table
 - 4.3 The appointments Table
 - 4.4 Entity Relationship Diagram
 - 4.5 Normalization (3NF) Explained
 - 4.6 Seed Data
- 5. Backend PHP API -- Complete File-by-File Walkthrough**
 - 5.1 Database Connection (db.php)
 - 5.2 Authentication Endpoints (auth/)
 - 5.3 Appointments CRUD Endpoints (appointments/)
 - 5.4 Lawyers Directory (lawyers/)
- 6. Critical SQL Operations (The 5 Required Techniques)**
 - 6.1 ACID Transactions + SELECT ... FOR UPDATE (Double-Booking Guard)
 - 6.2 Multi-Table SQL JOIN (Resolving Foreign Keys)
 - 6.3 Status State Machine (Server-Side Validation)
 - 6.4 Soft-Delete (Preserving Audit History)
 - 6.5 SQL Aggregates with GROUP BY (Dashboard Metrics)
- 7. Frontend React Application -- Architecture & Components**
 - 7.1 Project Setup (Vite + Tailwind + Material 3)
 - 7.2 API Layer (config.js, auth.js, appointments.js, lawyers.js)
 - 7.3 Authentication Context (AuthContext.jsx)
 - 7.4 Routing (App.jsx)
 - 7.5 Reusable Components
 - 7.6 Pages (All 11 Screens)
- 8. How Data Flows: A Complete Request Lifecycle**
- 9. How to Run the Project (Quickstart Guide)**
- 10. Test Accounts & Credentials**
- 11. Glossary of Terms for Beginners**

1. What is Lawyer Finder? (Project Overview)

Lawyer Finder is a database-driven web application that connects clients with verified lawyers based on their **specialization** (Corporate, Family, Criminal, etc.) and **location** (district in Bangladesh). It is built as a CSE 3104 Database Systems Lab project to demonstrate real-world database operations.

The application has **three modules**:

Module	User Role	What They Can Do
Client Module	client	Search for lawyers, view profiles, book consultations, manage their appointments (view, cancel).
Lawyer Module	lawyer	View incoming case requests, accept or reject appointments, mark cases as completed, view their statistics.
Admin Module	admin	View platform-wide statistics, see all appointments across all lawyers and clients, monitor system health.

For **Checkpoint 1**, the instructor requires: (1) the complete frontend built and running with all screens, and (2) one fully working database table demonstrated end-to-end. The **appointments** table was chosen because it has the richest CRUD operations, foreign-key relationships, and meaningful SQL features (transactions, locks, joins, aggregates) in the entire schema.

Tip: The entire project can be launched with a single command: `python run.py`. See Section 9 for details.

2. High-Level Architecture

The application follows a classic **three-tier architecture** -- the same pattern used by most modern web applications (Gmail, Facebook, Amazon). Think of it as three separate programs that talk to each other:

Layer	Technology	What It Does	Analogy
1. Frontend (Presentation)	React (Vite) Tailwind CSS Material Design 3	Renders the user interface. Handles button clicks, forms, and navigation. Sends HTTP requests to the backend.	The "face" of the app -- what users see and interact with.
2. Backend (Application)	Plain PHP PDO (MySQLi) Session Authentication	Receives HTTP requests. Validates input and enforces business rules. Executes SQL queries. Returns JSON responses.	The "brain" -- it decides what to do with the data.
3. Database (Data Storage)	MySQL 8.0 (XAMPP / InnoDB)	Stores all data permanently. Enforces referential integrity. Handles concurrent access safely.	The "memory" -- it remembers everything.

How they communicate (the request cycle):

1. The user clicks a button in the **React frontend** (e.g., 'Book Appointment').
2. React sends an **HTTP POST request** with JSON data to the PHP backend (e.g., POST /backend/appointments/create.php).
3. The **PHP backend** validates the data, opens a database transaction, and executes SQL queries against MySQL.
4. **MySQL** processes the SQL, acquires locks if needed, inserts/updates rows, and returns results.
5. PHP formats the result as a **JSON response** and sends it back to React.
6. React **updates the UI** (shows a success toast, refreshes the appointments list, etc.).

Important: The frontend and backend are **completely separate programs**. The React app runs on port 5173 (Vite dev server). The PHP backend runs on port 8000 (PHP built-in server). They communicate *only* through HTTP requests and JSON responses. This is called a **REST API** architecture.

3. Project Directory Structure (File Map)

Here is every file in the project with a one-line description:

Path	Purpose
run.py	One-command launcher that starts both backend and frontend servers
generate_pdf_guide.py	This script -- generates the PDF you are reading now
README.md	Project documentation with setup instructions
index.html	Original standalone HTML mockup with Material Design 3 tokens
backend/config/db.php	Database connection, CORS headers, session setup, JSON helpers
backend/database/schema.sql	MySQL schema with CREATE TABLE and seed INSERT statements
backend/auth/login.php	POST -- Authenticates user with email + password
backend/auth/register.php	POST -- Creates new user account (hashes password with bcrypt)
backend/auth/logout.php	POST -- Destroys PHP session
backend/auth/me.php	GET -- Returns currently logged-in user from session
backend/appointments/create.php	POST -- Books appointment (with transaction + lock)
backend/appointments/read.php	GET -- Lists appointments with multi-table SQL JOIN
backend/appointments/update.php	POST -- Updates status with state machine validation
backend/appointments/delete.php	POST -- Soft-cancels appointment (no hard delete)
backend/appointments/stats.php	GET -- Returns SQL aggregates (GROUP BY counts)
backend/lawyers/read.php	GET -- Searchable lawyer directory with filters
frontend/package.json	Node.js dependencies (React, React Router, Vite)
frontend/vite.config.js	Vite bundler configuration
frontend/tailwind.config.js	Tailwind CSS configuration with Material 3 color tokens
frontend/index.html	HTML entry point for the React SPA
frontend/src/main.jsx	React entry point -- mounts App into the DOM
frontend/src/App.jsx	Root component with all route definitions
frontend/src/index.css	Global styles, Material 3 tokens, animations
frontend/src/api/config.js	API base URL and universal fetch wrapper
frontend/src/api/auth.js	Auth API functions (login, register, logout, getMe)
frontend/src/api/appointments.js	Appointments API functions (create, read, update, delete, stats)
frontend/src/api/lawyers.js	Lawyers API functions (list, getByld, search)
frontend/src/context/AuthContext.jsx	Global auth state, role switcher, login/logout logic
frontend/src/components/TopBar.jsx	Navigation header with role switcher pill
frontend/src/components/Footer.jsx	Page footer with project info

frontend/src/components/LawyerCard.jsx	Reusable lawyer profile card
frontend/src/components/BookingModal.jsx	Modal dialog for booking appointments
frontend/src/components/CalendarPicker.jsx	Interactive date picker component
frontend/src/components/StatusBadge.jsx	Colored status pill (pending/accepted/etc.)
frontend/src/components/Toast.jsx	Notification toast popup

4. Database Schema Deep Dive (schema.sql)

The database is the foundation of the entire application. Everything -- users, lawyer profiles, appointments -- is stored as rows in MySQL tables. The schema file is located at backend/database/schema.sql.

4.1 The users Table

This table stores the login credentials and role for every person in the system -- whether they are a client, a lawyer, or an admin. Think of it as the 'address book' of the application.

SQL: CREATE TABLE users

```
CREATE TABLE users (
  id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('client','lawyer','admin') NOT NULL DEFAULT 'client',
  phone VARCHAR(30) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  INDEX idx_users_role (role),
  INDEX idx_users_email (email)
) ENGINE=InnoDB;
```

Column	Type	Explanation
id	INT UNSIGNED AUTO_INCREMENT	A unique number auto-assigned to each user. This is the Primary Key.
name	VARCHAR(120)	The user's full name (e.g., 'Sadia Anwar').
email	VARCHAR(150) UNIQUE	Login email. UNIQUE means no two users can share the same email.
password_hash	VARCHAR(255)	The bcrypt-hashed password. We NEVER store plain text passwords.
role	ENUM('client','lawyer','admin')	Controls what the user can do. Only these 3 values are allowed.
phone	VARCHAR(30) NULL	Optional phone number. NULL means it can be left empty.
created_at	TIMESTAMP	Automatically set when the row is first inserted.
updated_at	TIMESTAMP	Automatically updated every time the row is modified.

Tip: What is AUTO_INCREMENT? It means MySQL automatically generates the next number. If the last user had id=5, the next one gets id=6. You never have to set it manually.

4.2 The lawyers Table

This table stores the **professional profile** for users who have role='lawyer'. It has a **one-to-one relationship** with the users table via the user_id foreign key. This means every lawyer row points to exactly one user row.

SQL: CREATE TABLE lawyers

```
CREATE TABLE lawyers (
  id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id INT UNSIGNED NOT NULL UNIQUE,
  specialization VARCHAR(100) NOT NULL,
  district VARCHAR(80) NOT NULL,
  bio TEXT NULL,
  experience_years INT UNSIGNED NOT NULL DEFAULT 1,
```

```

bar_license VARCHAR(80) NULL,
consultation_fee DECIMAL(10,2) NOT NULL DEFAULT 1000.00,
rating DECIMAL(3,2) NOT NULL DEFAULT 4.80,
reviews_count INT UNSIGNED NOT NULL DEFAULT 0,
PRIMARY KEY (id),
INDEX idx_lawyers_spec_dist (specialization, district),
CONSTRAINT fk_lawyers_user
FOREIGN KEY (user_id) REFERENCES users(id)
ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

```

Column	Type	Explanation
user_id	INT UNSIGNED UNIQUE	Points to users(id). UNIQUE enforces one profile per lawyer. The FOREIGN KEY constraint m
specialization	VARCHAR(100)	Area of law: Corporate, Family, Criminal, Property, Tax, or Labor.
district	VARCHAR(80)	Geographic location: Dhaka, Chattogram, Sylhet, Khulna, Rajshahi.
consultation_fee	DECIMAL(10,2)	Fee in BDT. DECIMAL avoids floating-point rounding errors.
rating	DECIMAL(3,2)	Average rating (e.g., 4.90 out of 5.00).
bar_license	VARCHAR(80)	Official bar association license number.

Important: ON DELETE CASCADE means: if a user row is deleted from the users table, MySQL will *automatically* delete the corresponding lawyer row. This prevents 'orphan' records -- lawyer profiles that point to users that no longer exist.

4.3 The appointments Table

This is the **core transactional table** and the focus of Checkpoint 1. Every consultation booking creates one row here. It links a **client** (from users) to a **lawyer** (from lawyers) at a specific date and time.

SQL: CREATE TABLE appointments

```

CREATE TABLE appointments (
  id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  client_id INT UNSIGNED NOT NULL,
  lawyer_id INT UNSIGNED NOT NULL,
  appointment_date DATE NOT NULL,
  appointment_time TIME NOT NULL,
  case_description TEXT NOT NULL,
  status ENUM('pending','accepted','rejected',
    'completed','cancelled')
  NOT NULL DEFAULT 'pending',
  cancellation_reason VARCHAR(255) NULL,
  PRIMARY KEY (id),
  INDEX idx_appointments_lawyer_datetime
  (lawyer_id, appointment_date, appointment_time),
  CONSTRAINT fk_appointments_client
  FOREIGN KEY (client_id) REFERENCES users(id)
  ON DELETE CASCADE,
  CONSTRAINT fk_appointments_lawyer
  FOREIGN KEY (lawyer_id) REFERENCES lawyers(id)
  ON DELETE CASCADE
) ENGINE=InnoDB;

```

Column	Type	Explanation
client_id	INT UNSIGNED FK	Points to users(id) -- the person booking the consultation.
lawyer_id	INT UNSIGNED FK	Points to lawyers(id) -- the advocate being consulted.

appointment_date	DATE	The date of the consultation (e.g., 2026-08-20).
appointment_time	TIME	The time slot (e.g., 10:30:00 for 10:30 AM).
case_description	TEXT	Free-text description of the legal issue.
status	ENUM (5 values)	The current lifecycle state of the appointment.
cancellation_reason	VARCHAR(255) NULL	Optional reason stored when an appointment is cancelled.

Why the composite index?

The index `idx_appointments_lawyer_datetime` (`lawyer_id`, `appointment_date`, `appointment_time`) is a **composite index** -- it indexes three columns together. This is critical because our double-booking check query filters on exactly these three columns: 'Does this lawyer already have a booking at this date and time?' Without this index, MySQL would scan every row in the table. With it, the lookup is nearly instant.

4.4 Entity Relationship Diagram

The following table shows how the three tables relate to each other through foreign keys:

Relationship	Type	How It Works
users <-> lawyers	One-to-One	Each lawyer has exactly one user account. <code>lawyers.user_id</code> -> <code>users.id</code> (UNIQUE)
users <-> appointments	One-to-Many	Each client can have many appointments. <code>appointments.client_id</code> -> <code>users.id</code>
lawyers <-> appointments	One-to-Many	Each lawyer can have many appointments. <code>appointments.lawyer_id</code> -> <code>lawyers.id</code>

4.5 Normalization (3NF) Explained

Our schema is in **Third Normal Form (3NF)**. Here is what that means in simple terms:

1NF (First Normal Form): Every column contains a single value (no arrays or lists in a cell). For example, we don't store 'specialization1, specialization2' in one column.

2NF (Second Normal Form): Every non-key column depends on the *entire* primary key. Since all our tables use a single-column primary key (`id`), 2NF is automatically satisfied.

3NF (Third Normal Form): No non-key column depends on another non-key column. For example, we don't store the lawyer's name inside the appointments table -- we store `lawyer_id` and JOIN to get the name. This avoids data duplication (if the lawyer changes their name, we only update one row in users, not hundreds of appointment rows).

4.6 Seed Data

The schema file also inserts sample data so you can test immediately. Here are the pre-loaded accounts:

Name	Email	Role	Notes
System Admin	admin@lawyerfinder.com	admin	Platform administrator
Sadia Anwar	sadia@gmail.com	client	Default test client
Mahin Hasan	mahin@gmail.com	client	Second test client

Nusrat Tania	nusrat@gmail.com	client	Third test client
Farhan Ahmed	farhan@gmail.com	client	Fourth test client
Adv. Rahim Karim	rahim@lawyer.com	lawyer	Corporate law, Dhaka
Adv. Farzana Yasmin	farzana@lawyer.com	lawyer	Family law, Chattogram
Adv. Kamrul Hasan	kamrul@lawyer.com	lawyer	Criminal law, Sylhet
Adv. Nasrin Akter	nasrin@lawyer.com	lawyer	Property law, Khulna
Adv. Shafiul Alam	shafiul@lawyer.com	lawyer	Tax law, Rajshahi
Adv. Tania Rahman	tania@lawyer.com	lawyer	Labor law, Dhaka

Tip: All seed accounts use the password: password123 (stored as a bcrypt hash).

5. Backend PHP API -- Complete File-by-File Walkthrough

The backend is a collection of plain PHP scripts (no framework). Each script handles one API endpoint. The React frontend calls these scripts over HTTP to read and write data in the MySQL database.

5.1 Database Connection & Core Helpers (backend/config/db.php)

This is the first file loaded by every other PHP script (via `require_once`). It does four things:

- 1. Starts a PHP session** -- Sessions allow the server to 'remember' who is logged in across multiple requests. PHP assigns a unique cookie (PHPSESSID) to each browser.
- 2. Sets CORS headers** -- Since the React frontend (port 5173) and PHP backend (port 8000) run on different ports, the browser blocks requests by default (Same-Origin Policy). CORS headers explicitly tell the browser: 'Yes, requests from port 5173 are allowed.'
- 3. Connects to MySQL using PDO** -- PDO (PHP Data Objects) is PHP's built-in database abstraction layer. We configure it with: error mode `EXCEPTION` (throws errors instead of silent failures), fetch mode `ASSOC` (returns rows as associative arrays), and `emulate_prepares=false` (uses real MySQL prepared statements for security).
- 4. Defines helper functions** -- `send_json()` formats every API response consistently, `get_json_input()` reads the request body, and `get_auth_user()` checks the session.

Key Code: PDO MySQL Connection

```
// PDO Connection (from db.php)
$dsn = 'mysql:host=localhost;port=3306;dbname=lawyer_finder;charset=utf8mb4';
$pdo = new PDO($dsn, 'root', '', [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
]);
```

Key Code: send_json() Helper

```
// Standard JSON response helper
function send_json($success, $data, $message, $status_code = 200) {
    http_response_code($status_code);
    echo json_encode([
        'success' => (bool)$success,
        'data' => $data,
        'message' => (string)$message
    ]);
    exit;
}
```

Tip: Every API response has the same structure: `{success: true/false, data: ..., message: '...'}`. This makes it easy for the React frontend to handle responses consistently.

5.2 Authentication Endpoints (backend/auth/)

login.php (POST)

Accepts email and password. Looks up the user in the database using a LEFT JOIN with lawyers (so if the user is a lawyer, we also get their lawyer_id and specialization in one query). Verifies the password using PHP's password_verify() function, which compares the plain text password against the stored bcrypt hash. If valid, saves the user data to the PHP session and returns it as JSON.

Key Code: Password Verification

```
// Password verification (login.php)
$user = $stmt->fetch(); // fetch user row from database
if (!$user || !password_verify($password, $user['password_hash'])) {
    send_json(false, null, 'Invalid email or password.', 401);
}
$_SESSION['user'] = $userData; // save to session
send_json(true, $userData, 'Login successful!');
```

register.php (POST)

Creates a new user. Hashes the password with password_hash(PASSWORD_BCRYPT) before storing it. If the role is 'lawyer', it also creates a row in the lawyers table with default values.

logout.php (POST)

Destroys the PHP session and clears the session cookie.

me.php (GET)

Returns the currently logged-in user from the session. Used by the frontend on page load to restore login state.

5.3 Appointments CRUD Endpoints (backend/appointments/)

These are the most important files in the entire project because they demonstrate all five required SQL techniques. Each file is explained in detail in Section 6.

File	HTTP	SQL Features Demonstrated
create.php	POST	ACID Transaction, SELECT ... FOR UPDATE (row lock), INSERT, multi-table JOIN
read.php	GET	Multi-table INNER JOIN across 3 tables + 2 aliases, dynamic WHERE filters
update.php	POST	Status state machine validation, Transaction + lock for rescheduling
delete.php	POST	Soft-delete (UPDATE status='cancelled' instead of DELETE FROM)
stats.php	GET	COUNT(*), GROUP BY status, GROUP BY lawyer_id, LEFT JOIN aggregates

5.4 Lawyers Directory (backend/lawyers/read.php)

Returns a list of all lawyers, joined with their user profiles. Supports optional query parameters for filtering by specialization, district, and text search (LIKE queries). Used by the Search Lawyers page.

6. Critical SQL Operations (The 5 Required Techniques)

This section is the **most important part of the project** for your CSE 3104 submission. Each technique is explained with the actual SQL from the codebase.

6.1 ACID Transactions + SELECT ... FOR UPDATE

File: backend/appointments/create.php

The Problem: What happens if two clients try to book the same lawyer at the same time? Without protection, both requests could check 'is this slot free?' simultaneously, both see 'yes', and both insert a booking -- creating a double-booking disaster.

The Solution: We wrap the check-and-insert in a database **Transaction** and use SELECT ... FOR UPDATE to acquire an **exclusive row lock**.

Annotated Code: The Double-Booking Guard

```
// Step 1: Start a transaction
$pdo->beginTransaction();
// Step 2: Lock the time slot (FOR UPDATE = exclusive lock)
$conflict_sql = "
SELECT id FROM appointments
WHERE lawyer_id = :lawyer_id
AND appointment_date = :appointment_date
AND appointment_time = :appointment_time
AND status IN ('pending', 'accepted')
FOR UPDATE";
// Step 3: If a conflicting row exists, rollback
if ($existing_conflict) {
    $pdo->rollBack();
    send_json(false, null, 'Time slot already reserved.', 409);
}
// Step 4: Insert the new appointment
$insert_stmt->execute([...]);
// Step 5: Commit the transaction
$pdo->commit();
```

How FOR UPDATE works step by step:

1. **Client A** sends a booking request for Lawyer #1 on Aug 20 at 10:30 AM.
2. PHP starts a transaction and executes SELECT ... FOR UPDATE. MySQL finds no conflicting rows, but **locks the index range** so no other transaction can modify or insert into this slot.
3. **Client B** sends the same booking request 50 milliseconds later.
4. Client B's SELECT ... FOR UPDATE **blocks** (waits) because Client A holds the lock.
5. Client A inserts the appointment and commits. The lock is released.
6. Client B's SELECT now executes and **finds the conflicting row**. PHP rolls back and returns 409 Conflict.

This is called **Pessimistic Concurrency Control** -- we lock first, then check.

Important: **ACID** stands for: **A**tomicity (all or nothing), **C**onsistency (data stays valid), **I**solation (concurrent transactions don't interfere), **D**urability (committed data survives crashes).

6.2 Multi-Table SQL JOIN

File: backend/appointments/read.php

The Problem: The appointments table only stores foreign key IDs (client_id=2, lawyer_id=1). But the user wants to see names: 'Sadia Anwar booked Adv. Rahim Karim'. How do we resolve IDs to names?

The Solution: SQL JOIN. One query touches **three tables** (with the users table used **twice** -- once for the client's name, once for the lawyer's name):

SQL: Multi-Table INNER JOIN (3 tables, 4 aliases)

```
SELECT
a.id, a.appointment_date, a.status,
c.name AS client_name, -- from users (client)
c.email AS client_email,
u.name AS lawyer_name, -- from users (lawyer)
l.specialization, l.consultation_fee
FROM appointments a
INNER JOIN users c ON a.client_id = c.id -- client details
INNER JOIN lawyers l ON a.lawyer_id = l.id -- lawyer profile
INNER JOIN users u ON l.user_id = u.id -- lawyer's name
WHERE 1=1
ORDER BY a.appointment_date DESC;
```

Why use aliases? We reference the users table twice. To distinguish them, we alias one as c (client) and the other as u (the lawyer's user record). The JOIN chain is: appointments -> users (client) + appointments -> lawyers -> users (lawyer).

6.3 Status State Machine (Server-Side Validation)

File: backend/appointments/update.php

An appointment's status follows strict rules. You can't jump from 'pending' directly to 'completed' -- the lawyer must accept it first. The backend enforces these rules with a PHP array that maps each status to its allowed next states:

PHP Code: Status Transition Validation

```
$allowed_transitions = [
'pending' => ['accepted', 'rejected', 'cancelled'],
'accepted' => ['completed', 'cancelled', 'pending'],
'rejected' => ['pending'], // optional reopen
'completed' => [], // terminal - no changes allowed
'cancelled' => [], // terminal - no changes allowed
];
if (!in_array($new_status, $allowed_transitions[$curr_status])) {
    send_json(false, null,
    "Cannot change from '{$curr_status}' to '{$new_status}'.", 400);
}
```

Current Status	Allowed Next States	Who Can Do It
pending	accepted, rejected, cancelled	Lawyer accepts/rejects; Client cancels
accepted	completed, cancelled	Lawyer marks done; Either party cancels
rejected	pending (reopen)	Admin can reopen
completed	(none -- terminal)	No further changes allowed
cancelled	(none -- terminal)	No further changes allowed

6.4 Soft-Delete (Preserving Audit History)

File: backend/appointments/delete.php

Hard delete (DELETE FROM appointments WHERE id = ?) permanently removes the row. The data is gone forever.

Soft delete (what we use) updates status = 'cancelled' and stores the reason. The row remains in the database for auditing, reporting, and compliance.

SQL: Soft-Delete Query

```
// Soft-cancel (NOT hard delete)
UPDATE appointments
SET status = 'cancelled',
    cancellation_reason = :reason,
    updated_at = NOW()
WHERE id = :id;
```

Tip: Soft-delete is a best practice in production systems. Imagine a lawyer disputes that a client cancelled -- with soft-delete, we have the complete history. With hard-delete, the evidence is gone.

6.5 SQL Aggregates with GROUP BY

File: backend/appointments/stats.php

The admin and lawyer dashboards need summary statistics: 'How many pending cases does Lawyer #1 have?' Instead of fetching all rows and counting in PHP, we let MySQL do the counting with aggregate functions:

SQL: GROUP BY Aggregate Query

```
-- Lawyer-specific: count appointments by status
SELECT status, COUNT(*) as count
FROM appointments
WHERE lawyer_id = :lawyer_id
GROUP BY status;
-- Result example:
-- | status | count |
-- |-----|-----|
-- | pending | 3 |
-- | accepted | 5 |
-- | completed | 12 |
```

SQL: Specialization Breakdown (Admin Dashboard)

```
-- Platform-wide: appointments per specialization
SELECT l.specialization, COUNT(a.id) as appointment_count
FROM lawyers l
LEFT JOIN appointments a ON l.id = a.lawyer_id
GROUP BY l.specialization;
```

Why LEFT JOIN here? If a lawyer has zero appointments, an INNER JOIN would exclude them. LEFT JOIN keeps all lawyers and shows 0 for those with no bookings.

7. Frontend React Application

7.1 Project Setup (Vite + Tailwind CSS + Material Design 3)

The frontend is a **Single-Page Application (SPA)** built with React. That means the browser loads one HTML file, and JavaScript dynamically swaps content as you navigate -- no full-page reloads.

Technology	What It Does	Config File
Vite	Ultra-fast dev server and bundler. Compiles JSX/CSS on the fly.	vite.config.js
React 18	Component-based UI library. Each page/widget is a reusable 'component'.	package.json
React Router v6	Client-side routing. Maps URL paths (/lawyers, /admin/dashboard) to components.	App.jsx
Tailwind CSS	Utility-first CSS framework. Write styles as class names (e.g., 'rounded-tall-60').	tailwind.config.js
Material Design 3	Google's design system. We use its color tokens, typography, and elevation system.	index.css

7.2 API Layer (frontend/src/api/)

The API layer is a set of JavaScript files that wrap HTTP fetch calls. This keeps API URLs and request logic in one place instead of scattered across components.

config.js -- The Universal Fetch Wrapper

Contains apiRequest(endpoint, options) which: (1) prepends the base URL, (2) sets JSON headers, (3) includes session cookies (credentials: 'include'), (4) tries the primary server (port 8000), and if that fails, (5) falls back to XAMPP Apache (port 80). This means the frontend works whether you use run.py or standard XAMPP.

Simplified Code: API Fallback Pattern

```
// API request with automatic fallback (config.js)
export async function apiRequest(endpoint, options = {}) {
  try {
    return await tryFetch('http://localhost:8000');
  } catch (err) {
    return await tryFetch('http://localhost/lawyer-finder-api');
  }
}
```

API File	Functions Exported	Backend Endpoints Called
auth.js	login(), register(), logout(), getMe()	/auth/login.php, /auth/register.php, etc.
appointments.js	create(), getAll(), update(), cancel(), getStats()	/appointments/create.php, read.php, etc.
lawyers.js	getAll(), getById(), search()	/lawyers/read.php

7.3 Authentication Context (AuthContext.jsx)

React **Context** is a way to share data across all components without passing props manually. AuthContext stores: the current user object, login/logout functions, and role detection helpers (isClient, isLawyer, isAdmin).

Key Feature: 1-Click Role Switcher

For Checkpoint 1 demonstration, the TopBar includes a role switcher pill that instantly switches between Client, Lawyer, and Admin views -- without logging out and logging in again. This uses the `switchRole(role)` function which swaps the mock user in state and `localStorage`.

Code: The Role Switcher Function

```
// Role switcher (AuthContext.jsx)
const switchRole = (role) => {
  let mock;
  if (role === 'lawyer') {
    mock = { id: 6, name: 'Adv. Rahim Karim', role: 'lawyer', ... };
  } else if (role === 'admin') {
    mock = { id: 1, name: 'System Admin', role: 'admin', ... };
  } else {
    mock = { id: 2, name: 'Sadia Anwar', role: 'client', ... };
  }
  setUser(mock);
  localStorage.setItem('lawyer_finder_user', JSON.stringify(mock));
};
```

7.4 Routing (App.jsx)

React Router maps URL paths to React components. Here are all routes defined in `App.jsx`:

URL Path	Component	Module
/	LandingPage	Public
/login	LoginPage	Public
/register	RegisterPage	Public
/lawyers	SearchLawyersPage	Public
/lawyers/:id	LawyerProfilePage	Public
/book/:id	BookAppointmentPage	Public
/client/appointments	MyAppointmentsPage	Client
/client/profile	ClientEditProfilePage	Client
/lawyer/requests	IncomingRequestsPage	Lawyer
/lawyer/profile	LawyerEditProfilePage	Lawyer
/admin/dashboard	AdminDashboardPage	Admin

7.5 Reusable Components (frontend/src/components/)

Component	What It Renders	Used By
TopBar.jsx	Navigation header with logo, nav links, role switcher pill, and login/logout buttons	Every page (via App.jsx)
Footer.jsx	Page footer with university name, project info, and copyright.	Every page (via App.jsx)
LawyerCard.jsx	A card displaying a lawyer's name, specialization, rating, fee, and a 'Quick Book' button	SearchLawyersPage, LandingPage
BookingModal.jsx	A modal dialog with date picker, time slot selector, and case description field	LawyerProfilePage, SearchLawyersPage
CalendarPicker.jsx	An interactive calendar widget for selecting appointment dates	BookingModal
StatusBadge.jsx	A small colored pill showing the appointment status (green=accepted, yellow=pending, red=rejected)	MyAppointmentsPage, IncomingRequestsPage, AdminDashboard
Toast.jsx	A notification popup that appears at the bottom of the screen for short messages	Used only triggered by AuthContext.showToast()

7.6 Pages (All 11 Screens)

Public Module Pages:

LandingPage.jsx (/) -- The homepage. Features a hero section with search bar, trust statistics (animated counters), curated lawyer cards, service categories with icons, and an architecture diagram showing the 3-tier stack. This is the first thing visitors see.

SearchLawyersPage.jsx (/lawyers) -- A full-featured search page. Users can filter lawyers by specialization (clickable chips), district (dropdown), and sort by rating. Each result renders as a LawyerCard with a 'Quick Book' button that opens the BookingModal.

LawyerProfilePage.jsx (/lawyers/:id) -- A detailed profile page showing the lawyer's bio, credentials, bar license, experience, fee, rating, and reviews. Includes an embedded calendar booking widget.

BookAppointmentPage.jsx (/book/:id) -- A dedicated booking form page (alternative to the modal). Includes date picker, time slot grid, and case description textarea.

LoginPage.jsx (/login) & RegisterPage.jsx (/register) -- Authentication forms with Material 3 styled inputs, validation feedback, and role selection (client or lawyer).

Client Module Pages:

MyAppointmentsPage.jsx (/client/appointments) -- Shows all appointments for the logged-in client. Each appointment card shows the lawyer's name, date, time, status (via StatusBadge), and a Cancel button. Cancellation opens a confirmation dialog that calls delete.php (soft-cancel). Data is fetched from read.php with ?client_id=X filter.

ClientEditProfilePage.jsx (/client/profile) -- Profile editing form for clients.

Lawyer Module Pages:

IncomingRequestsPage.jsx (/lawyer/requests) -- The lawyer's case management dashboard. Shows aggregate statistics at the top (total cases, pending, accepted, completed -- fetched from stats.php). Below, lists all incoming appointment requests with Accept/Reject/Complete action buttons. Each action calls update.php with the appropriate status transition.

LawyerEditProfilePage.jsx (/lawyer/profile) -- Profile editing form for lawyers.

Admin Module Pages:

AdminDashboardPage.jsx (/admin/dashboard) -- Platform-wide overview. Shows total clients, total lawyers, total appointments, and appointments this week (from stats.php without lawyer_id filter). Displays a master appointments table with all bookings across the platform, filterable by status. Includes a specialization breakdown chart.

8. How Data Flows: A Complete Request Lifecycle

Let's trace what happens when a client books an appointment, step by step:

Step	Layer	What Happens
1	User	Client clicks 'Book Appointment' on LawyerProfilePage and fills in the date, time, and case description.
2	React	BookingModal calls appointmentsApi.create({ lawyer_id: 1, appointment_date: '2026-08-20', appointment_time: '10:30', case_description: '...' })
3	API Layer	config.js wraps the data in a JSON POST request and sends it to http://localhost:8000/backend/appointments/create.php
4	PHP Backend	create.php receives the JSON body, validates all fields, and verifies both the client and lawyer exist in the database.
5	MySQL (Transaction)	PHP calls \$pdo->beginTransaction(). MySQL creates an isolated workspace.
6	MySQL (Lock)	PHP executes SELECT ... FOR UPDATE. MySQL acquires an exclusive lock on the lawyer's time slot.
7	MySQL (Insert)	No conflict found. PHP executes INSERT INTO appointments. MySQL writes the new row.
8	MySQL (Commit)	PHP calls \$pdo->commit(). MySQL makes the changes permanent and releases the lock.
9	PHP Backend	create.php fetches the new appointment with a JOIN query (to get client name, lawyer name, etc.) and calls send_json(true, \$data, 'Success')
10	React	The Promise resolves. BookingModal shows a success toast, closes the modal, and navigates to MyAppointmentsPage.
11	React	MyAppointmentsPage calls appointmentsApi.getAll({client_id: 2}), which hits read.php. The new appointment appears in the list with status 'Booked'.

Tip: This entire cycle -- from user click to database write to UI update -- typically takes less than 200 milliseconds.

9. How to Run the Project (Quickstart Guide)

Prerequisites:

Software	Version	Why Needed	Download
XAMPP	8.x+	Provides PHP and MySQL	apachefriends.org
Node.js	18+	Runs the React dev server	nodejs.org
Python	3.8+	Runs the launcher script	python.org

Step-by-Step Instructions:

Step 1: Open XAMPP Control Panel and start **MySQL** (Apache is optional -- we use PHP's built-in server).

Step 2: Open a terminal/command prompt in the project root folder.

Step 3: Install frontend dependencies (first time only):

```
cd frontend && npm install && cd ..
```

Step 4: Run the project:

```
python run.py
```

What run.py does automatically:

1. Finds PHP at C:\xampp\php\php.exe (or in your PATH)
2. Checks if the lawyer_finder database exists -- if not, imports schema.sql
3. Starts the PHP backend server on **http://localhost:8000**
4. Starts the Vite React dev server on **http://localhost:5173**
5. Opens your browser automatically
6. Press **Ctrl+C** to stop both servers cleanly.

Alternative: Manual Setup (without run.py)

Terminal 1 (Backend): C:\xampp\php\php.exe -S localhost:8000 -t backend/

Terminal 2 (Frontend): cd frontend && npm run dev

Database: Open phpMyAdmin, create database 'lawyer_finder', import backend/database/schema.sql

10. Test Accounts & Credentials

Use these accounts to test all three modules. All passwords are: password123

Role	Email	Password	What To Test
Client	sadia@gmail.com	password123	Search lawyers, book appointments, view My Appointments, cancel bookings
Lawyer	rahim@lawyer.com	password123	View Incoming Requests, accept/reject cases, see stats dashboard
Admin	admin@lawyerfinder.com	password123	View platform stats, see all appointments, filter by status

Tip: You can also use the **Role Switcher pill** in the top navigation bar to instantly switch between Client, Lawyer, and Admin views without logging out.

Suggested Testing Workflow:

1. Open the app (<http://localhost:5173>) -- you land on the homepage as 'Sadiah Anwar' (client).
2. Click 'Find Lawyers' -> browse the directory -> click a lawyer -> book an appointment.
3. Go to 'My Appointments' -- see your new booking with status 'pending'.
4. Switch to Lawyer view (click the role switcher) -> go to 'Incoming Requests' -> Accept the booking.
5. Switch back to Client -> go to 'My Appointments' -> status is now 'accepted'.
6. Switch to Admin -> go to 'Dashboard' -> see the appointment in the master list.
7. Try cancelling an appointment from the Client view -- the soft-delete stores the reason.

11. Glossary of Terms for Beginners

Term	Definition
API	Application Programming Interface -- a set of rules for how two programs communicate. Our PHP backend is an API that the React frontend uses to communicate with the database.
ACID	Atomicity, Consistency, Isolation, Durability -- four guarantees that database transactions provide.
AUTO_INCREMENT	A MySQL feature that automatically assigns the next integer value to a column when a new row is inserted.
bcrypt	A password hashing algorithm. It converts 'password123' into a long random-looking string. It's one-way (you can't reverse it).
CORS	Cross-Origin Resource Sharing -- browser security mechanism. Without proper CORS headers, the browser blocks requests from other domains.
CRUD	Create, Read, Update, Delete -- the four basic database operations.
ENUM	A MySQL column type that restricts values to a fixed list (e.g., only 'pending', 'accepted', 'rejected', 'completed', 'cancelled').
Foreign Key (FK)	A column that references the primary key of another table. It creates a link between two tables.
GROUP BY	A SQL clause that groups rows with the same value and lets you apply aggregate functions (COUNT, SUM, AVG) to each group.
InnoDB	MySQL's default storage engine. It supports transactions, row-level locking, and foreign key constraints.
JOIN	A SQL operation that combines rows from two or more tables based on a related column (usually a foreign key).
JSON	JavaScript Object Notation -- a lightweight data format used for API communication. Example: {"name": "Sadiah", "role": "client"}
JSX	JavaScript XML -- React's syntax for writing HTML-like code inside JavaScript files.
Normalization (3NF)	Database design technique to eliminate data redundancy. Instead of repeating data, we use foreign keys to reference shared data.
PDO	PHP Data Objects -- PHP's built-in library for connecting to databases securely using prepared statements.
Primary Key (PK)	A column (or set of columns) that uniquely identifies each row in a table. Usually 'id'.
Prepared Statement	A SQL query with placeholders (:name) instead of raw values. Prevents SQL injection attacks.
REST API	Representational State Transfer -- an architectural style where each URL endpoint represents a resource and HTTP methods (GET, POST, PUT, DELETE) are used to interact with it.
SELECT ... FOR UPDATE	A MySQL query that reads rows AND locks them, preventing other transactions from modifying them until the lock is released.
Session	A way for the server to remember who is logged in. PHP stores session data on the server and identifies the user by a cookie (PHPSESSID).
Soft-Delete	Instead of removing a database row permanently (DELETE), we set a flag (status='cancelled') so the data is preserved for audit purposes.
SPA	Single-Page Application -- a web app that loads one HTML page and dynamically updates content using JavaScript (no full-page reloads).
SQL Injection	A security attack where malicious SQL is inserted through user input. Prevented by using prepared statements with parameterized queries.
Transaction	A sequence of SQL operations that either ALL succeed (commit) or ALL fail (rollback). Ensures data consistency.
Vite	A modern JavaScript build tool. Much faster than Webpack. Serves React files with hot-module replacement during development.

--- End of Lawyer Finder Codebase Guide ---

Generated automatically by generate_pdf_guide.py using ReportLab.